

# AWS Cost Monitoring & Budget Alerts

**Project:** AWS Cost Monitoring & Budget Alerts

**Services:** AWS Budgets, Cost Explorer, Lambda, SNS, SES, CloudWatch

**Description:** Monitor AWS spending and generate automated budget alerts using AWS Budgets, SNS, Lambda, and SES. The system sends notifications when spending thresholds are exceeded and logs events in CloudWatch.

**Skills:** Cost monitoring, serverless automation, SNS notifications, Lambda scripting, cloud governance.

**Definition:** The AWS Cost Monitoring & Budget Alerts project is designed to monitor AWS cloud spending in real time and notify administrators whenever predefined budget thresholds are reached. It uses AWS Budgets to track usage costs, SNS for notifications, Lambda for automated event processing, SES for email communication, and CloudWatch for logging and monitoring.

**Scope of the Project:** The scope of this project is to design and implement an automated AWS cost monitoring system that tracks spending, triggers alerts, and logs notification events using serverless AWS services.

**This project covers:**

- Automated Budget Monitoring using AWS Budgets.
- Threshold-based alerts using SNS and SES.
- Serverless event processing using AWS Lambda.
- Logging and monitoring using CloudWatch.
- Cost governance and financial monitoring for AWS resources.

**Methodology:**

## 1. AWS Budgets + SNS + Lambda - Best Method

- Uses AWS Budgets, SNS, Lambda, SES, and CloudWatch.
- Automatically triggers notifications when spending exceeds thresholds.
- Serverless and fully automated solution.
- Provides real-time budget governance and monitoring.
- Cost-effective and suitable for free-tier learning projects.

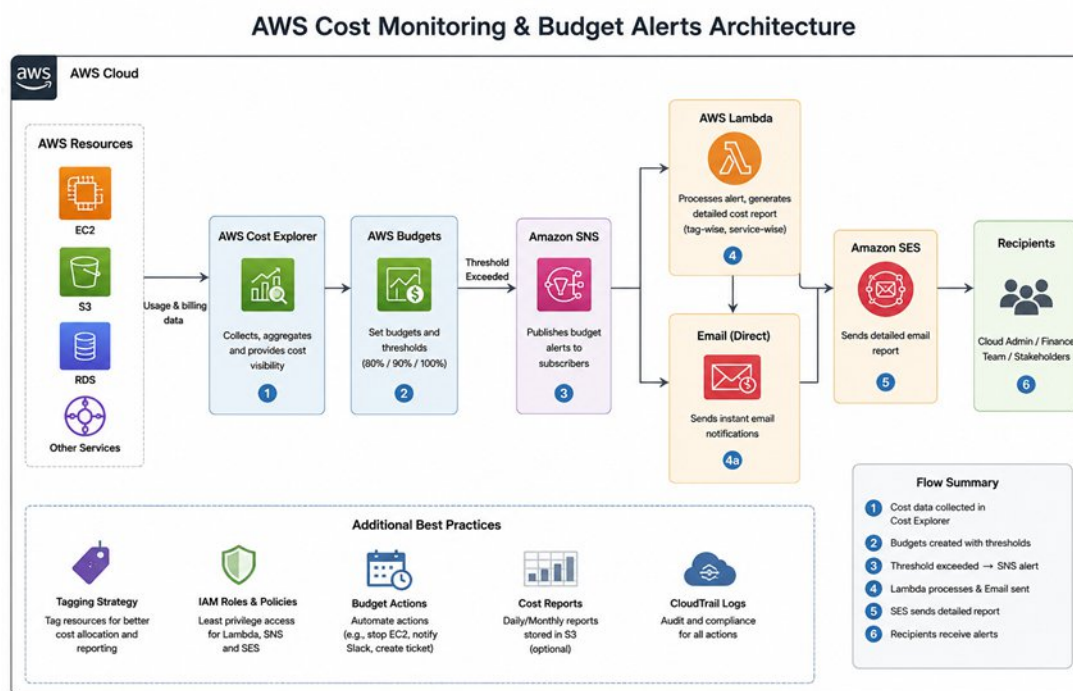
## 2. Cost Explorer + Manual Monitoring – Alternative Method

- Uses AWS Cost Explorer dashboards for monitoring usage.
- Allows visualization of service-wise AWS costs.
- Requires manual checking of spending trends.
- Suitable for small environments and learning purposes.

## AWS Services used in the Project:

1. **AWS Budgets** Tracks AWS spending and generates alerts when predefined thresholds are reached.
2. **Amazon SNS** Sends notifications through email and triggers Lambda functions.
3. **AWS Lambda** Processes SNS notifications and logs budget events.
4. **Amazon SES** Handles verified email notifications.
5. **Amazon CloudWatch** Stores Lambda execution logs and monitoring data.
6. **AWS Cost Explorer** Provides visual dashboards and cost analysis.

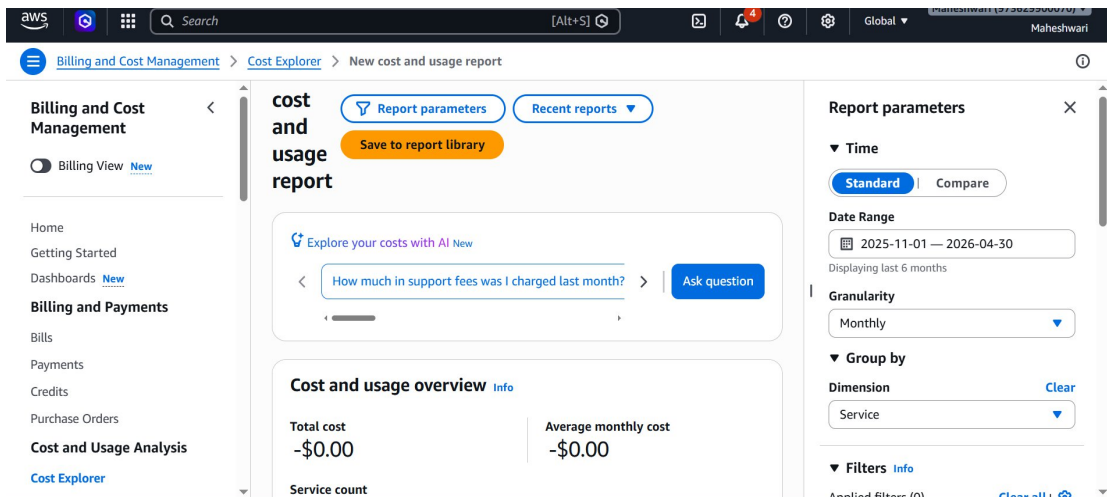
## Architecture Diagram:



## Implementation steps

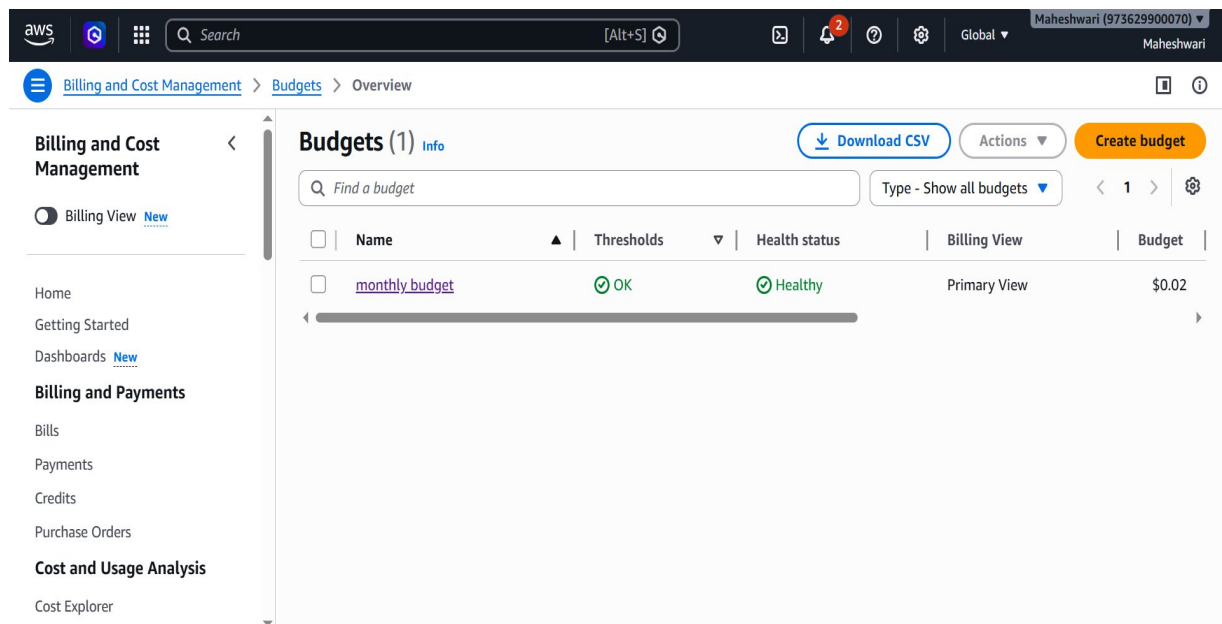
### Step 1: Enable Cost Explorer

- Go to AWS Billing Dashboard.
- Enable AWS Cost Explorer.
- Wait for activation.



## Step 2: Create Budget

- Go to Billing – Budgets.
- Create Custom Cost Budget.
- Set budget amount and thresholds.
- Add email notifications.



## Step 3: Create SNS Topic

- Go to SNS – Topics – Create Topic.
- Select Standard Topic.
- Topic Name: CostAlertTopic.
- Create email subscription and confirm from Gmail.

aws

Search

[Alt+S]

2

Asia Pacific (Mumbai)

Maheshwari (973629900070)

Maheshwari

Amazon SNS

Topics

CostAlertTopic

Subscription: c9de8719-25f8-4a0b-84c8-72e60a2a4204

Amazon SNS

Dashboard

Topics

Subscriptions

▼ Mobile

Push notifications

Text messaging (SMS)

Subscription: c9de8719-25f8-4a0b-84c8-72e60a2a4204

EditDelete

Details

ARN  
arn:aws:sns:ap-south-1:973629900070:CostAlertTopic:c9de8719-25f8-4a0b-84c8-72e60a2a4204

Status  
Confirmed

Endpoint  
arn:aws:lambda:ap-south-1:973629900070:function:CostAlertFunction

Protocol  
LAMBDA

Topic  
CostAlertTopic

Subscription filter policy

Redrive policy (dead-letter queue)

Subscription filter policy

This policy filters the messages that a subscriber receives.

No filter policy configured for this subscription.  
To apply a filter policy, edit this subscription.

Edit

aws

Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:  
arn:aws:sns:ap-south-1:973629900070:CostAlertTopic:fef0116e-87ac-4ecb-95e7-f33a0d59dff7

If it was not your intention to subscribe, [click here to unsubscribe](#).

aws

Search

[Alt+S]

2

Asia Pacific (Mumbai)

Maheshwari (973629900070)

Maheshwari

Amazon SNS

Topics

CostAlertTopic

Amazon SNS

Dashboard

Topics

Subscriptions

▼ Mobile

Push notifications

Text messaging (SMS)

CostAlertTopic

EditDeletePublish message

Details

Name  
CostAlertTopic

ARN  
arn:aws:sns:ap-south-1:973629900070:CostAlertTopic

Display name  
-

Type  
Standard

Topic owner  
973629900070

Subscriptions

Access policy

Delivery policy (HTTP/S)

Delivery status logging

Encryption

Tags

Integrations

Subscriptions (2)

EditDeleteRequest confirmationConfirm subscriptionCreate subscription

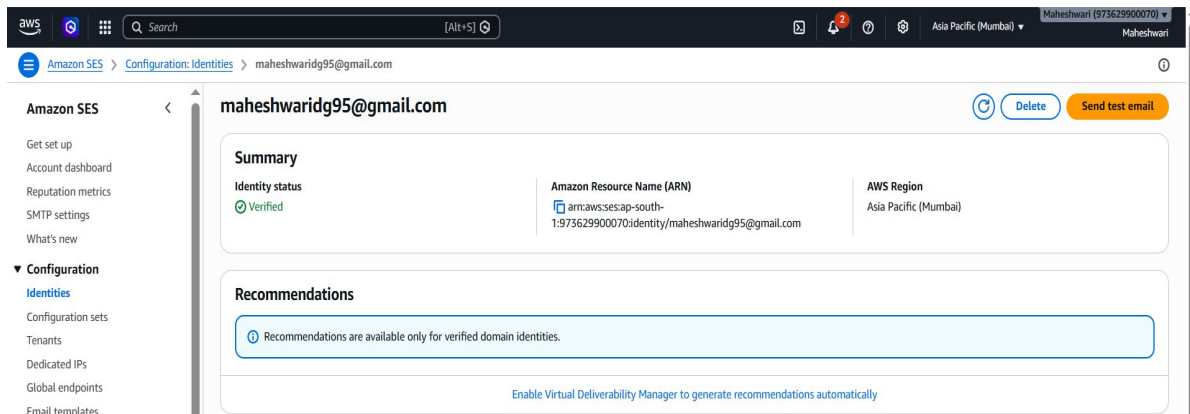
Search

< 1 >

| ID                                   | Endpoint                                   | Status    | Protocol |
|--------------------------------------|--------------------------------------------|-----------|----------|
| c9de8719-25f8-4a0b-84c8-72e60a2a4204 | arn:aws:lambda:ap-south-1:973629900070:... | Confirmed | LAMBDA   |
| fef0116e-87ac-4ecb-95e7-f33a0d59dff7 | maheshwaridg95@gmail.com                   | Confirmed | EMAIL    |

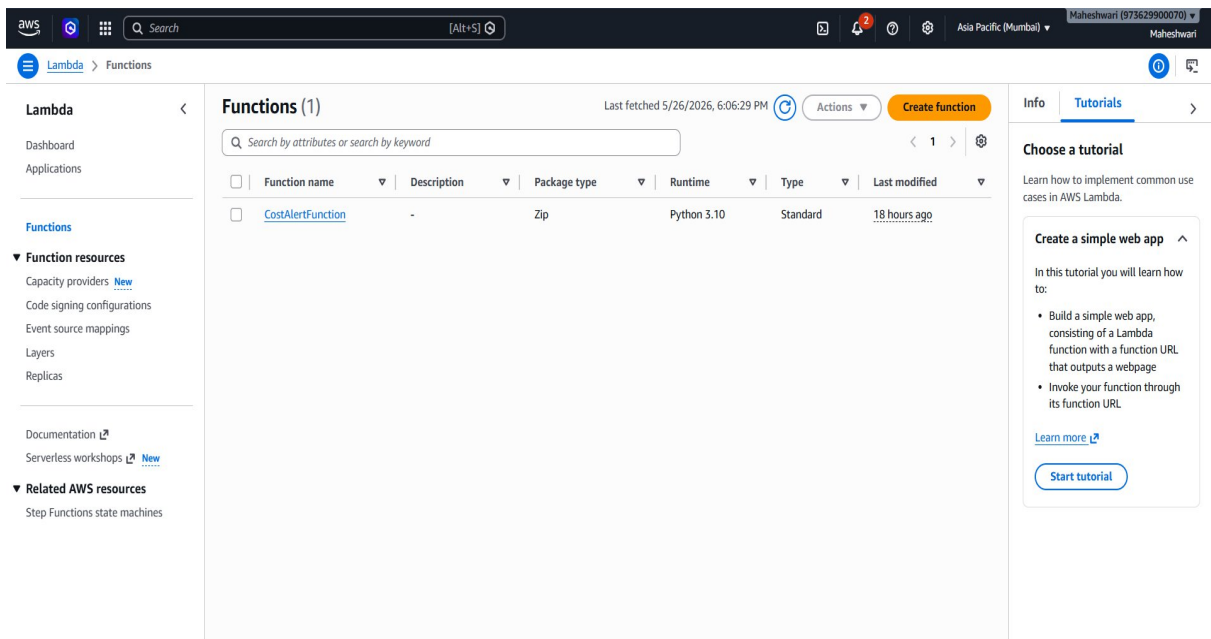
## Step 4: Create SES

- Go to SES → Configuration → Create Identities.
- Select Email address.
- Add Email address



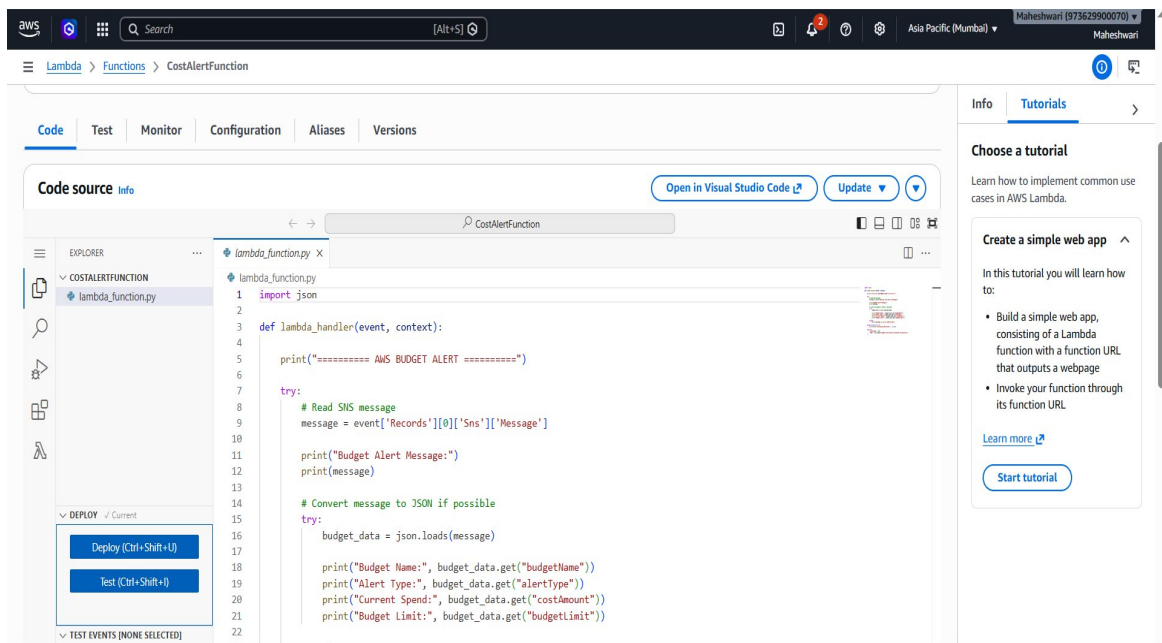
## Step 5: Create Lambda Function

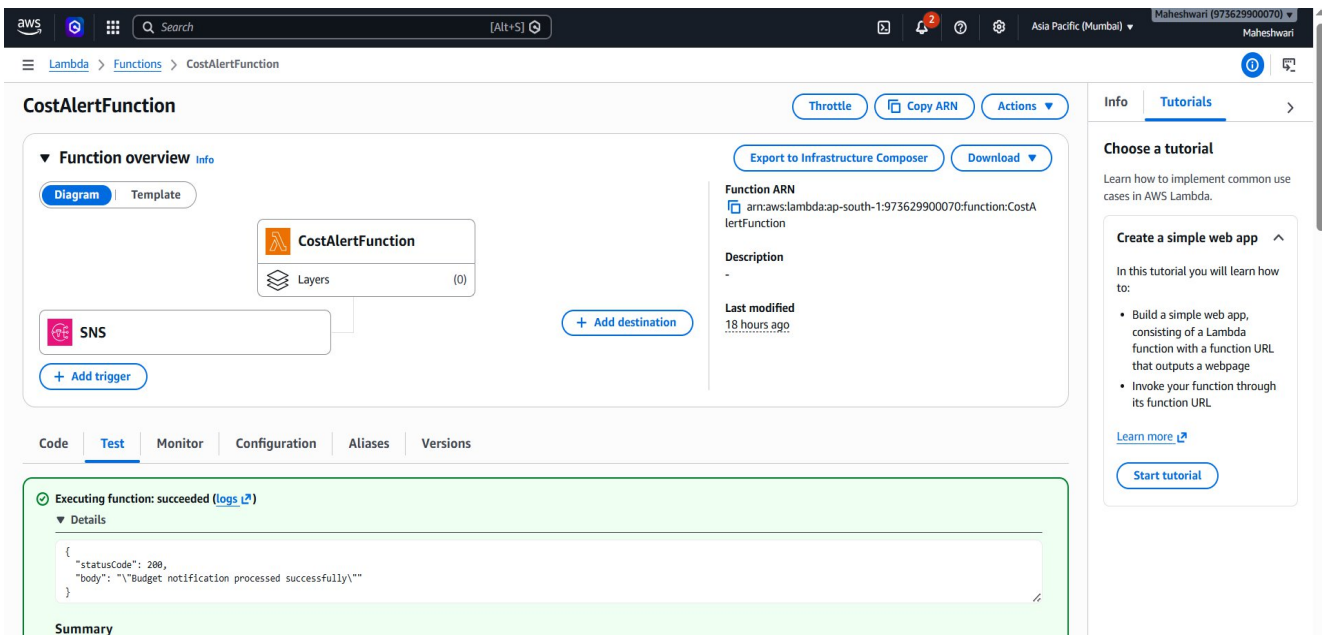
- Go to Lambda → Create Function.
- Runtime: Python 3.10.
- Function Name: CostMonitoringFunction.
- Attach basic Lambda permissions.



## Step 6: Add Lambda Code and Test

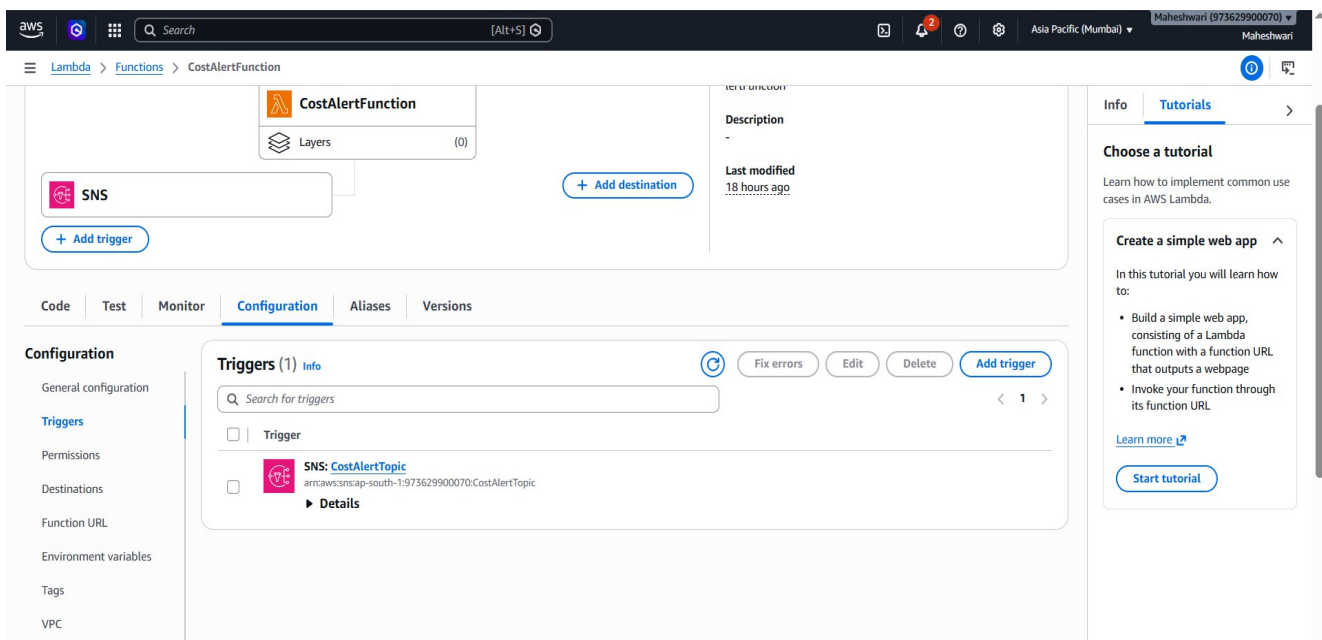
- Paste the Python monitoring code.
- Click Deploy.
- Create test event and run the function.





## Step 7: Connect SNS to Lambda

- Open Lambda → Add Trigger.
- Select SNS.
- Choose CostAlertTopic.
- Save the trigger configuration.



## Step 8: Verify Monitoring

- Publish test SNS message.
- Verify Lambda execution.
- Check CloudWatch logs.
- Verify email alerts in Gmail.

- Sample Lambda Code:

```
import json
```

```
def lambda_handler(event, context):
```

```
    print ("===== AWS BUDGET ALERT =====")
```

```
    try:
```

```
        message = event['Records'][0]['Sns']['Message']
```

```
        print(message)
```

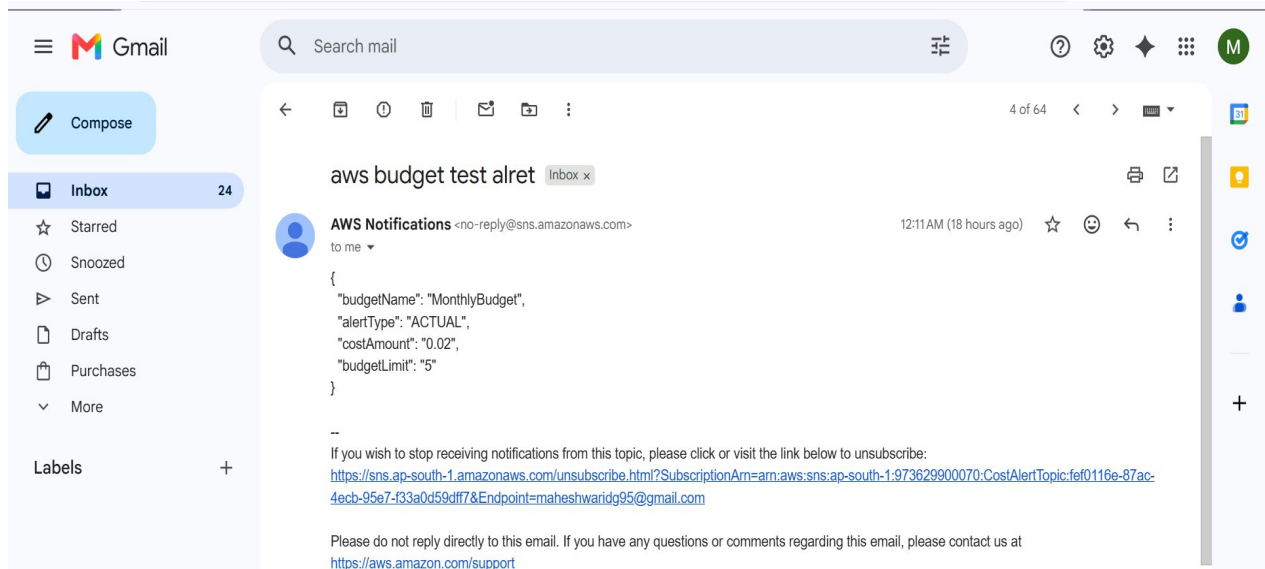
```
    except Exception as e:
```

```
        print ("Error:", str(e))
```

```
    return {
```

```
        'statusCode': 200
```

```
    }
```



CloudWatch > Log management > /aws/lambda/CostAlertFunction > All events

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom Local timezone Display

| Timestamp                     | Message                                                                         | Log stream name                                     |
|-------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------|
| 2026-05-26T00:11:48.845+05:30 | "budgetName": "MonthlyBudget",                                                  | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | "alertType": "ACTUAL",                                                          | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | "costAmount": "0.02",                                                           | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | "budgetLimit": "5"                                                              | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | }                                                                               | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | Budget Name: MonthlyBudget                                                      | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | Alert Type: ACTUAL                                                              | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | Current Spend: 0.02                                                             | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.845+05:30 | Budget Limit: 5                                                                 | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.847+05:30 | END RequestId: bdfdc6e5-507e-411c-ab5f-347886d797d8                             | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:11:48.847+05:30 | REPORT RequestId: bdfdc6e5-507e-411c-ab5f-347886d797d8 Duration: 1.47 ms B11... | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:13:12.739+05:30 | START RequestId: 73bdfc4c-49d2-4b07-8659-d29ad6be74eb Version: \$LATEST         | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:13:12.740+05:30 | ***** AWS BUDGET ALERT *****                                                    | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:13:12.740+05:30 | Error processing SNS event: 'Records'                                           | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |
| 2026-05-26T00:13:12.743+05:30 | END RequestId: 73bdfc4c-49d2-4b07-8659-d29ad6be74eb                             | 2026/05/25/[LATEST]463f88931d3446a5b5c998ed4fa0390a |

Back to top

## Troubleshooting:

### Issue 1: SNS Subscription Pending

**Solution:** Open Gmail and confirm the SNS subscription email.

### Issue 2: Lambda Not Triggering

**Solution:** Verify SNS trigger is attached to the Lambda function.

### Issue 3: No CloudWatch Logs

**Solution:** Check Lambda execution role permissions and rerun the test event.

## Conclusion:

The AWS Cost Monitoring & Budget Alerts project provides an automated and serverless approach for monitoring AWS cloud spending. Using AWS Budgets, SNS, Lambda, SES, and CloudWatch, organizations can track expenses, receive proactive notifications, and improve cloud financial governance. The project is cost-effective, beginner-friendly, and suitable for real-world cloud monitoring use cases.